

Human:

I am taking a deep learning class. Your job is to help me fill the gap in my knowledge from lecture. Specifically, I want you to help me understand the KV cache in more detail.

I have attached relevant lecture notes that talk about transformers.

Here are my questions:

1. What is the KV cache?

2. What specifically is cached?

3. How does prefilling and autoregressive generation work with the KV cache?

Claude:

Understanding the KV Cache in Transformers

Based on your lecture notes, I'll explain how the KV cache builds on the attention mechanism you've learned. Let me connect this to the concepts from your lectures.

1. What is the KV Cache?

The **KV cache** is a memory optimization technique used during autoregressive text generation. It stores previously computed Key (K) and Value (V) matrices so they don't need to be recomputed at every generation step.

From your notes, you learned that attention computes:

$$\text{attention score} = \text{softmax}(QK^T / \sqrt{d}) \times V$$

The key insight is: **when generating token by token, the K and V matrices for previous tokens never change** — only the new token's Q, K, V need to be computed.

2. What Specifically is Cached?

For each transformer layer, the cache stores:

- **Keys (K)**: The key projections for all previously generated tokens
- **Values (V)**: The value projections for all previously generated tokens

I like how it specifies exactly what is cached. It makes the distinction between K vs the weight matrix for the keys.

Specifically, if you have a sequence of length t tokens, the cache contains:

- $K = [k_1, k_2, \dots, k_t]$ where each $k_i = W_k \cdot x_i$
- $V = [v_1, v_2, \dots, v_t]$ where each $v_i = W_v \cdot x_i$

The Query (Q) for the new token is computed fresh each time and is NOT cached.

3. Prefilling and Autoregressive Generation with KV Cache

Your notes mention two phases that are relevant here:

Prefilling Phase (Processing the prompt)

When you first input a prompt like "Give me the book", the model:

1. Processes ALL tokens at once in parallel

2. Computes Q, K, V for every token in the prompt

3. Stores all K and V matrices in the cache

4. Outputs the first generated token

Example with prompt "Give me the book":

Tokens: [Give] [me] [the] [book]

Compute: Q, K, V for all 4 tokens simultaneously

Cache: K_1, K_2, K_3, K_4 and V_1, V_2, V_3, V_4

Autoregressive Generation Phase

Your notes show this process where you "set initial conditions for decoder" and generate one token at a time. With KV cache:

- Step 1: Generate token 5 (e.g., "■■■")
- Compute only Q_5, K_5, V_5 for the new token
 - Retrieve cached K_1, K_2, K_3, K_4 and V_1, V_2, V_3, V_4
 - Compute attention: $\text{softmax}(Q_5 \cdot [K_1, \dots, K_5]^T) \cdot [V_1, \dots, V_5]$

- Add K_5, V_5 to cache

Step 2: Generate token 6 (e.g., "■■■■■■")

- Compute only Q_6, K_6, V_6
- Retrieve cached $K_1...K_5$ and $V_1...V_5$
- Compute attention: $\text{softmax}(Q_6 \cdot [K_1, \dots, K_6]^T) \cdot [V_1, \dots, V_6]$
- Add K_6, V_6 to cache

...and so on until you hit <stop>

Why This Matters

Without KV cache:

- Token 100 would require computing Q, K, V for all 100 tokens
- Complexity: $O(n^2)$ where n = sequence length

With KV cache:

- Token 100 only computes $Q_{100}, K_{100}, V_{100}$
- Retrieves the other 99 K, V pairs from cache
- Complexity: $O(n)$ for each new token

This is especially important for the bottleneck issue your notes mention — you need to "capture all of the input sentence in this bottleneck." The KV cache helps make long-sequence generation tractable.

Connection to Your Notes: This relates to the "teacher forcing" training vs. inference distinction in your Lecture 18 notes. During training, you have the full sequence and can compute everything in parallel. During inference (autoregressive generation), you generate one token at a time, which is where KV cache becomes crucial for efficiency.

Human:

When computing K and V, don't we need to use the hidden state of the model? Thus, how can we save the K and the V of each token when it changes depending on the new tokens?

Claude: I really like how claude is fast to the point and says that it is because of maked attention.

Excellent question! You've identified what seems like a paradox. Let me clarify how this works — it's all about **masked (causal) attention**.

The Key Insight: Causal Masking

Looking at your **Lecture 19 notes (page 2)**, you can see the Transformer architecture with both encoder and decoder. For autoregressive generation (like GPT), we use **decoder-only models with masked self-attention**.

The crucial point: In **masked attention**, token i can **ONLY** attend to tokens 1 through i , never to future tokens ($i+1, i+2, \dots$)

Why K and V Don't Change

Let's trace through what happens:

When Processing Token 3:

Token 1: $h_1 = f(x_1, \text{attends to: token 1 only})$
 Token 2: $h_2 = f(x_2, \text{attends to: tokens 1,2})$
 Token 3: $h_3 = f(x_3, \text{attends to: tokens 1,2,3})$

From these hidden states:

$K_1 = W_k \cdot h_1$
 $K_2 = W_k \cdot h_2$
 $K_3 = W_k \cdot h_3$

When Processing Token 4:

Token 1: $h_1 = f(x_1, \text{attends to: token 1 only}) \leftarrow \text{UNCHANGED!}$
Token 2: $h_2 = f(x_2, \text{attends to: tokens 1,2}) \leftarrow \text{UNCHANGED!}$
Token 3: $h_3 = f(x_3, \text{attends to: tokens 1,2,3}) \leftarrow \text{UNCHANGED!}$
Token 4: $h_4 = f(x_4, \text{attends to: tokens 1,2,3,4}) \leftarrow \text{NEW!}$

K_1, K_2, K_3 remain the same!
Only K_4 is newly computed.

Why h_1, h_2, h_3 don't change:

- h_1 was computed only looking at token 1
- h_2 was computed only looking at tokens 1-2
- h_3 was computed only looking at tokens 1-3
- When we add token 4, the mask **prevents** tokens 1-3 from seeing it

Visualizing the Attention Mask

From your Lecture 18 notes (page 3), you can see the GPT-3 architecture mentions "masked attention." Here's what the attention matrix looks like:

	k_1	k_2	k_3	k_4	
q_1	✓	✗	✗	✗	(token 1 can only attend to itself)
q_2	✓	✓	✗	✗	(token 2 can attend to 1,2)
q_3	✓	✓	✓	✗	(token 3 can attend to 1,2,3)
q_4	✓	✓	✓	✓	(token 4 can attend to 1,2,3,4)

The ✗ positions get $-\infty$ added before softmax, making them effectively 0.

Contrast with Encoder (Bidirectional Attention)

You're absolutely right that in a **bidirectional encoder** (like BERT), the hidden states WOULD change:

Encoder with tokens [A, B]:

h_A attends to [A, B]

h_B attends to [A, B]

Add token C:

h_A attends to [A, B, C] $\leftarrow$ CHANGES!

h_B attends to [A, B, C] $\leftarrow$ CHANGES!

h_C attends to [A, B, C] $\leftarrow$ NEW!

This is why **KV cache only works for autoregressive/decoder models**, not for encoders.

Summary

The KV cache works because:

1. **Causal masking** ensures previous tokens' hidden states never see future tokens
2. Once h_i is computed, it's frozen — it will never be recomputed
3. Therefore $K_i = W_k \cdot h_i$ and $V_i = W_v \cdot h_i$ are also frozen
4. Only the NEW token needs fresh Q, K, V computation

Your intuition was correct for bidirectional models, but the autoregressive constraint (which your notes reference with the "unrolled length of encoder and decoder might be different" comment) is what makes caching possible!

Human:

The formula for K is $K = \text{hidden state} \cdot \text{weight}$. The hidden state changes during autoregressive generation. Thus, doesn't K change too?

Claude:

Claude answered this question really well. This was a real question I had and was confused about.

Great question! You're pushing on exactly the right point. Let me clarify with a concrete example of what the "hidden state" actually is during generation.

The Critical Detail: Position-Specific Hidden States

The key is that **each position has its own hidden state**, and that hidden state doesn't change once computed (due to causal masking).

Concrete Example - Layer by Layer

Let's say we're generating and currently have tokens: ["Give", "me", "the"]

At generation step 3 (have 3 tokens):

```
Layer 0 (Input Embeddings + Positional Encoding):
├─ Position 1:  $e_1$  (embedding of "Give")
├─ Position 2:  $e_2$  (embedding of "me")
└─ Position 3:  $e_3$  (embedding of "the")

Layer 1 (First Transformer Block):
├─ Position 1:  $h_1^{(1)} = e_1 + \text{Attention}(e_1, \text{can only see } e_1) + \text{FFN}$ 
├─ Position 2:  $h_2^{(1)} = e_2 + \text{Attention}(e_2, \text{can only see } e_1, e_2) + \text{FFN}$ 
└─ Position 3:  $h_3^{(1)} = e_3 + \text{Attention}(e_3, \text{can only see } e_1, e_2, e_3) + \text{FFN}$ 

From these hidden states:
 $K_1^{(1)} = W_k \cdot h_1^{(1)}$ 
 $K_2^{(1)} = W_k \cdot h_2^{(1)}$ 
 $K_3^{(1)} = W_k \cdot h_3^{(1)}$ 

These are CACHED ✓
```

At generation step 4 (adding "book"):

```
Layer 0:
├─ Position 1:  $e_1$  (same as before)
├─ Position 2:  $e_2$  (same as before)
├─ Position 3:  $e_3$  (same as before)
└─ Position 4:  $e_4$  (NEW - embedding of "book")

Layer 1:
├─ Position 1:  $h_1^{(1)} = e_1 + \text{Attention}(e_1, \text{sees } e_1) \leftarrow \text{IDENTICAL TO BEFORE!}$ 
├─ Position 2:  $h_2^{(1)} = e_2 + \text{Attention}(e_2, \text{sees } e_1, e_2) \leftarrow \text{IDENTICAL TO BEFORE!}$ 
├─ Position 3:  $h_3^{(1)} = e_3 + \text{Attention}(e_3, \text{sees } e_1, e_2, e_3) \leftarrow \text{IDENTICAL TO BEFORE!}$ 
└─ Position 4:  $h_4^{(1)} = e_4 + \text{Attention}(e_4, \text{sees } e_1, e_2, e_3, e_4) \leftarrow \text{NEW!}$ 

 $K_1^{(1)}, K_2^{(1)}, K_3^{(1)}$  are retrieved from cache (unchanged!)
Only  $K_4^{(1)} = W_k \cdot h_4^{(1)}$  needs to be computed
```

Why h_1, h_2, h_3 Don't Change

The crucial insight: **h_1 at position 1 doesn't change because:**

- It's computed from e_1
- Plus attention over positions that position 1 can see (only position 1 itself)
- Position 1 cannot see position 4 due to the causal mask
- Therefore, adding token 4 has zero effect on h_1

Same logic for h_2 (can only see positions 1-2) and h_3 (can only see positions 1-3).

What You Might Be Thinking Of

You might be thinking of the hidden state as a **single vector that accumulates information** (like in an RNN):

```
❌ WRONG (RNN-style thinking):
 $h_1 = f(x_1)$ 
 $h_2 = f(x_2, h_1) \leftarrow \text{depends on } h_1$ 
 $h_3 = f(x_3, h_2) \leftarrow \text{depends on } h_2$ 
 $h_4 = f(x_4, h_3) \leftarrow \text{changes everything before it!}$ 
```

But Transformers are different - they use **position-specific representations**:

```
✓ CORRECT (Transformer with causal mask):  
h1 = f(x1, attends to {x1})  
h2 = f(x2, attends to {x1, x2})  
h3 = f(x3, attends to {x1, x2, x3})  
h4 = f(x4, attends to {x1, x2, x3, x4})  
  
Each hi is independent once computed!
```

Looking at Your Notes

In your **Lecture 18 notes (page 1)**, you can see the encoder-decoder architecture where the encoder has stacked RNN blocks, and the decoder has RNN blocks. But in the **Lecture 19 notes**, you see the Transformer architecture which is fundamentally different - it doesn't have recurrent connections.

The key difference:

I like how it references the lecture notes as well since it ties it back to the source material.

- **RNN:** h_t depends on h_{t-1} , so everything changes
- **Transformer:** h_i depends only on positions $\leq i$ through attention (not through recurrence)

Bottom Line

You're right that $K = W_k \times \text{hidden_state}$, but:

- The hidden state **at position** i (h_i) does not change when we add tokens at positions $> i$
- This is guaranteed by causal masking
- Therefore $K_i = W_k \times h_i$ also doesn't change
- We can safely cache it!

Does this clarify why the hidden states at previous positions remain frozen?

Human:

So if I have tokens "give me the" as the first input, we get hidden state H for layer l , where H is $d_{\text{model}} \times 3$. Then I pass "give me the book" in a fresh model, we get hidden state H' , where H' is $d_{\text{model}} \times 4$. So would the first 3 rows of H' be equal to H ?

Claude:

I asked this question to confirm what I was thinking and I loved the immediate response by Claude.

YES! Exactly right! This is the KEY property that makes KV caching work.

The Answer: YES, the first 3 columns are identical

If you have:

- H of shape $(d_{\text{model}} \times 3)$ for "give me the"
- H' of shape $(d_{\text{model}} \times 4)$ for "give me the book"

Then: $H'[:, 0:3] == H$ (the first 3 columns of H' are identical to H)

This is guaranteed by the causal masking structure.

Why This Works

```
Processing "give me the" in isolation:
Position 1 ("give"):  $h_1 = f(\text{attends to position 1 only})$ 
Position 2 ("me"):  $h_2 = f(\text{attends to positions 1,2})$ 
Position 3 ("the"):  $h_3 = f(\text{attends to positions 1,2,3})$ 
 $\rightarrow H = [h_1, h_2, h_3]$ 

Processing "give me the book" from scratch:
Position 1 ("give"):  $h_1' = f(\text{attends to position 1 only})$ 
Position 2 ("me"):  $h_2' = f(\text{attends to positions 1,2})$ 
Position 3 ("the"):  $h_3' = f(\text{attends to positions 1,2,3})$ 
Position 4 ("book"):  $h_4' = f(\text{attends to positions 1,2,3,4})$ 
 $\rightarrow H' = [h_1', h_2', h_3', h_4']$ 

Since the attention mask prevents positions 1-3 from seeing position 4:
 $h_1' = h_1 \quad \checkmark$ 
 $h_2' = h_2 \quad \checkmark$ 
 $h_3' = h_3 \quad \checkmark$ 
```

This IS the KV Cache Optimization

What you just described is exactly what KV caching exploits:

Without KV cache (naive approach):

```
Step 1: Process "give me the"  $\rightarrow$  compute H, extract K, V
Step 2: Process "give me the book"  $\rightarrow$  compute H' from scratch
        (wastefully recomputing the same first 3 columns!)
```

With KV cache (smart approach):

```
Step 1: Process "give me the"  $\rightarrow$  compute H, extract K, V, CACHE them
Step 2: Only process "book"  $\rightarrow$  compute only  $h_4$ , extract  $K_4, V_4$ 
        Retrieve  $K_1, K_2, K_3, V_1, V_2, V_3$  from cache
        Concatenate:  $[K_1, K_2, K_3, K_4]$  and  $[V_1, V_2, V_3, V_4]$ 
```

Verification This Works at ALL Layers

This property holds **at every transformer layer**:

```
Layer 1:
 $H_1(\text{first 3 cols}) \text{ for "give me the"} == H_1'(\text{first 3 cols}) \text{ for "give me the book"}$ 

Layer 2:
 $H_2(\text{first 3 cols}) \text{ for "give me the"} == H_2'(\text{first 3 cols}) \text{ for "give me the book"}$ 

...and so on for all N layers
```

Because each layer maintains the causal structure.

Important Note on "Fresh Model"

Your phrase "fresh model" is important. You mean:

- Running the forward pass from scratch with the longer sequence
- NOT training a new model (weights W_q, W_k, W_v are the same)

If you literally trained two different models, then no, they'd be different. But with the **same trained model**, processing prefixes will always give identical hidden states regardless of what comes after (due to causality).

Summary

You've discovered the fundamental property that makes KV caching mathematically correct:

Causality Property: For any sequence prefix, the hidden states (and therefore K, V) computed for that prefix are **invariant** to what tokens come after it.

This is why we can:

1. Compute K, V for "give me the"
2. Cache them
3. Only compute K, V for "book"
4. Concatenate cached + new
5. Get the exact same result as if we'd processed "give me the book" from scratch

Great insight connecting the dimensions to the caching property!

Human:

What is the softmax attention formula with masking?

Claude:

Great question! Let me show you the formula with masking added.

Basic Attention (from your Lecture 18 notes)

```
Attention(Q, K, V) = softmax(QK^T / √d) · V
```

Attention WITH Masking

```
Attention(Q, K, V) = softmax((QK^T / √d) + M) · V
```

Where **M** is the mask matrix.

Here, I liked how it references the formula from lecture notes. This makes it easy to compare the formulas.

The Mask Matrix M

For causal (autoregressive) attention, the mask matrix looks like this:

```
M = [  0   -∞   -∞   -∞ ]
     [  0    0   -∞   -∞ ]
     [  0    0    0   -∞ ]
     [  0    0    0    0 ]
```

For a sequence of length 4, where:

- 0 means "allowed to attend"
- -∞ means "not allowed to attend" (masked out)

Why -∞?

While I was reading this, I wanted to ask why it would add a mask instead of the standard way of multiplying, but I like how it explained it in the same response.

When you add -∞ before softmax:

```
Before softmax: scores = [2.3, 1.5, -∞, -∞]
```

```
After softmax:
```

```
e^2.3 / (e^2.3 + e^1.5 + e^(-∞) + e^(-∞))
```

```
= e^2.3 / (e^2.3 + e^1.5 + 0 + 0)
```

```
The -∞ positions become 0 in the attention weights!
```

Step-by-Step Example

Let's say we have 3 tokens and are computing attention for all positions:

Step 1: Compute $QK^T / \sqrt{d}$

```
QK^T / √d = [ 0.8   0.6   0.4 ]
              [ 0.5   0.9   0.3 ]
              [ 0.7   0.2   1.0 ]
```

Step 2: Add Mask M

```
Mask M = [  0   -∞   -∞  ]
          [  0    0   -∞  ]
          [  0    0    0  ]

(QK^T / √d) + M = [ 0.8   -∞   -∞  ]
                  [ 0.5   0.9   -∞  ]
                  [ 0.7   0.2   1.0  ]
```

Step 3: Apply Softmax (row-wise)

```
Row 1: softmax([0.8, -∞, -∞]) = [1.0, 0.0, 0.0]
Row 2: softmax([0.5, 0.9, -∞]) = [0.4, 0.6, 0.0] (approximately)
Row 3: softmax([0.7, 0.2, 1.0]) = [0.3, 0.2, 0.5] (approximately)

Attention weights = [ 1.0  0.0  0.0 ]
                   [ 0.4  0.6  0.0 ]
                   [ 0.3  0.2  0.5 ]
```

Step 4: Multiply by V

```
Output = Attention_weights · V
```

Relating to Your Notes

Looking at your **Lecture 18 notes (page 7)**, you have the cross-attention formula where "Keys + values from encoder, Queries from decoder."

For **self-attention with masking** (decoder-only models like GPT), all Q, K, V come from the same sequence, but the mask prevents positions from attending to future positions.

Common Implementations

In practice, you'll see two equivalent forms:

Form 1 (additive mask):

```
scores = (Q @ K.T) / sqrt(d)
scores = scores + mask # mask has 0 and -inf
attn_weights = softmax(scores)
output = attn_weights @ V
```

Form 2 (multiplicative mask):

```
scores = (Q @ K.T) / sqrt(d)
scores = scores.masked_fill(mask == 0, float('-inf'))
attn_weights = softmax(scores)
output = attn_weights @ V
```

Both achieve the same result!

Summary

The complete masked attention formula is:

```
Attention(Q, K, V, M) = softmax((QK^T / √d) + M) · V
```

Where M is structured so that position i can only attend to positions ≤ i, which is what enables KV caching to work correctly!